

```
{  
    return contents;  
}  
public synchronized void put( int value )  
{  
    contents = value;  
}  
}
```

Note that methods `get()` and `put()` are each synchronized. If my class were calling the `put()` method, then nobody could call either the `put()` or the `get()` method in that instance. I have locked up that instance and it's hands off for everybody until I'm done. Because the class `LockBox` contains a synchronized method, every instance of class `LockBox` would have a special lock associated with it.

So, you see: when somebody is putting a value to the `contents` variable, nobody else can get it until the `put` is done. Likewise, when somebody is getting the value currently in the `contents` variable, nobody else can change that value in the middle of the read.

In other words, if an object has more than one synchronized method, then only one of those synchronized methods at a time can be executing. Synchronized methods are like having a single bathroom—no matter how many need it, it can only be used by one person—boy or girl—at a time. When the synchronized method is done executing, it releases the lock. The monitor gives the go ahead to the highest-priority thread that is ready to execute the synchronized method. If a synchronized method gets its turn to execute but then—during that execution—it finds it cannot proceed ( maybe I/O is blocked or there's some other hold up) —then the thread may voluntarily call its method `wait()`. In the `wait()` state, the thread doesn't take up processor resources and doesn't hold a lock on the monitor.

When a thread executing a synchronized method completes, the thread can execute the `notify()` method. Executing the `notify()` method tells any waiting threads to become ready so the scheduler can get them back running again. The `notify()` method is like calling:

“Head up, you're on deck.”

After a thread has been notified, it can attempt to get a lock on the object. Similarly, if a synchronized method no longer needs its lock for some reason, it can call `notifyAll()`, which alerts all waiting threads that they are free to enter the ready mode and again attempt to get a lock on the object.